LEVEL 2 PROJECT REPORT

CI/CD Pipeline Automation System

Automated testing and Docker image publishing with GitHub Actions

Repository	GitHub Repository
Docker image	Docker Hub: sapnaahamed/cicd-pipeline-automation
Workflow runs	GitHub Actions

Prepared as a practical DevOps project demonstrating CI automation, containerization, and image publishing.

1. Project Overview

This project implements a basic CI/CD pipeline for a Python Flask web application. GitHub Actions runs automated tests when code is pushed or a pull request targets the main branch. When tests pass on a push to main, the workflow builds a Docker image and publishes it to Docker Hub.

2. Objectives

• Create a small Flask application with health-check and home endpoints.
• Verify application behavior using automated unit tests.
• Package the application as a Docker image.
• Automate test execution and Docker image publishing with GitHub Actions.
• Verify that the published image can be pulled and run in a container.

3. Technology Stack

Technology	Purpose
Python + Flask	Web application and HTTP endpoints
unittest	Automated application tests
Docker	Container image creation and runtime verification
GitHub Actions	CI workflow orchestration
Docker Hub	Remote storage for the published image

4. Application Endpoints

Endpoint	Purpose	Observed result
/	Returns a basic application response	HTTP 200; JSON response
/health	Health-check endpoint	HTTP 200; JSON response

5. CI/CD Workflow

The configured GitHub Actions workflow follows this sequence:

1. Code is pushed or a pull request is opened against **main**.
2. The workflow installs dependencies and runs the unit tests.
3. If the tests succeed on a push to **main**, the workflow builds the Docker image.
4. GitHub Actions authenticates to Docker Hub using repository secrets.
5. The image is pushed to Docker Hub with the **latest** tag.

6. Validation and Results

The GitHub Actions run completed successfully for both the test job and the build-and-push job. The published image was then pulled and launched in a temporary GitHub Codespaces environment. Requests to the home endpoint and the health endpoint returned JSON responses, and the container logs showed successful HTTP 200 responses.

Container verification commands

```
docker pull sapnaahamed/cicd-pipeline-automation:latest
docker run -d --name cicd-app -p 5000:5000
sapnaahamed/cicd-pipeline-automation:latest
curl http://localhost:5000/health
curl http://localhost:5000/
```

7. Repository Structure

app.py — Flask application
requirements.txt — Python dependency list
test_app.py — unit tests
Dockerfile — container build instructions
.dockerignore — files excluded from the Docker build context
.github/workflows/ci-cd.yml — CI/CD workflow
screenshots/ — evidence screenshots

8. Security Practices

Docker Hub credentials are stored as GitHub repository secrets rather than hard-coded into the workflow. A Docker Hub access token is used for authentication. Secrets should not be printed in workflow logs or committed to the repository.

9. Limitations and Future Improvements

The project automates testing and image publishing. Running the image was verified manually in a temporary Codespaces environment; the project does not currently include a permanent public deployment, automated deployment to a hosting platform, or continuous runtime monitoring. Future improvements could add deployment automation, image version tags, vulnerability scanning, and uptime monitoring.

10. Project Links and Evidence

GitHub repository:
https://github.com/sapnaahamed23-hub/cognevance_cicd-pipeline-automation

GitHub Actions runs:
https://github.com/sapnaahamed23-hub/cognevance_cicd-pipeline-automation/actions

Screenshots folder: https://github.com/sapnaahamed23-hub/cognevance_cicd-pipeline-automation/tree/main/screenshots

Docker Hub image: https://hub.docker.com/r/sapnaahamed/cicd-pipeline-automation

Evidence screenshots included in the repository:

• github-repository-files.png — repository files
• github-actions-pipeline-success.png — successful workflow run
• dockerhub-image-latest.png — published Docker image
• docker-container-runtime-success.png — successful container runtime verification

11. Conclusion

This project demonstrates a practical CI/CD workflow for a containerized Flask application. Automated tests act as a quality gate, and successful main-branch builds publish the Docker image to Docker Hub. Runtime checks confirmed that the published image could be executed and that its endpoints responded successfully.

Prepared by: Sapna Ahamed